

Python高级应用

数据可视化

1 案例背景

2 知识准备

3 任务要求

4 任务解析

现在社会是一个高速发展的社会，科技发达，信息流通，人们之间的交流越来越密切，生活也越来越方便，大数据就是这个高科技时代的产物，在当前这个以数据为主导的世界中，各种各样的用户正在以多种方式收集数据，每一个人都想从他们所拥有的数据中了解更多具有价值的信息来辅助企业或者个人的管理；数据可视化以图形或图表形式显示数据信息，使用户更容易理解和分析数据。随着大数据时代的到来，企业的财务数据量堆积，密密麻麻的数据的可读性较差让财务人员在分析财务数据的时候难以抓住重点。财务人员需要掌握将数据转化为可视化的能力，把传统的用表格或数字展现财务数据的方式，替换为将财务数据用图片和线条以更加直观的方式展现出来，使数据更加客观、更具说服力，数据可视化更能够适应不同使用者的需要，辅助管理层实现快速决策。

数据可视化是python最常见的应用领域之一，Python可调用大量可视化库，实现财务数据的可视化，因此掌握可视化库的使用将成为必备的技能。

财务人员经常需要做的财务分析有那些主题，需要那些数据做分析，不同的分析主题用什么图表展示效果最佳？

1 案例背景

2 知识准备

3 任务要求

4 任务解析

python的可视化库有很多，下面列举几个最常用的介绍一下。

► Matplotlib

Matplotlib是一个基于python的2D画图库，能够用python脚本方便的画出折线图，直方图，功率谱图，散点图等常用图表；

Matplotlib是Python中众多数据可视化库的鼻祖，其设计风格与20世纪80年代设计的商业化程序语言MATLAB十分接近，具有很多强大且复杂的可视化功能，经过十几年它任然是Python使用者最常用的画图库。由于 matplotlib 是第一个 Python 可视化程序库，有许多别的程序库都是建立在它的基础上或者直接调用它。比如pandas和Seaborn就是matplotlib的外包，它们让你能用更少的代码去调用 matplotlib的方法。Matplotlib包含多种类型的API，可以采用多种方式绘制图表并对图表进行定制，是最基础的底层数据可视化第三方库，语言风格简单、易懂，特别**适合初学者**入门学习。

➤ Seaborn

Seaborn是在matplotlib的基础上进行了更高级的API封装，从而使得作图更加容易，在大多数情况下使用seaborn能做出很具有吸引力的图，而使用matplotlib就能制作具有更多特色的图。应该把Seaborn视为matplotlib的补充，而不是替代物。

➤ Pyecharts

pyecharts是一款将python与echarts结合的强大的数据可视化工具，生成的图表精巧，交互性良好，可轻松集成至 Flask, Sanic, Django 等主流 Web 框架，得到众多开发者的认可。

➤ bokeh

bokeh是一个面向web浏览器的交互式可视化库，它提供了多功能图形的优雅、简洁的构造，并在大型数据集或流式数据集上提供高性能的交互性

用matplotlib绘制可视化图表，主要有3种接口形式：

- Plt接口，例如常用的plt.plot()，它是matplotlib的一个state-based交互接口，相关操作不面向特定的实例对象，而是面向"当前"图。
- 面向对象接口，这里的面向对象主要是指Figure和Axes两类对象。Figure提供了容纳多个Axes的画板，而Axes则是所有图标数据、图例配置等绘图形元素的容器。面向对象的绘图，就是通过调用Figure或Axes两类实例的方法完成绘图的过程。通俗的说，就是将plt中的图形赋值给一个Figure或Axes实例，方便后续调用操作。
- pylab接口，其引入了numpy和pyplot的所有接口，自然也可用于绘制图表，仍然可看做是pyplot接口形式，因其过于庞大官方不建议使用。

Pyplot 是 Matplotlib 的子库，调用matplotlib库绘图一般是用pyplot子模块，其集成了绝大部分常用方法接口，提供了和 MATLAB 类似的绘图 API。Pyplot 是常用的绘图模块，能很方便让用户绘制 2D 图表，Pyplot 包含一系列绘图函数的相关函数，每个函数会对当前的图像进行一些修改，例如：给图像加上标记，生新的图像，在图像中产生新的绘图区域等等。使用的时候，我们可以使用 import 导入 pyplot 库，并设置一个别名 plt，后续用该别名来引用Pyplot包的方法。

引入Pyplot包

```
#引入matplotlib.pyplot 库  
import matplotlib.pyplot as plt
```

函数 plot() 函数是绘制二维图形的最基本函数，该函数用于画图时，可以绘制点和线。

plot() 函数使用语法

画单条线

```
plot([x], y, [fmt], *, data=None, **kwargs)
```

画多条线

```
plot([x], y, [fmt], [x2], y2, [fmt2], ..., **kwargs)
```

plot() 函数常用参数说明：

- x, y: 点或线的节点，x 为 x 轴数据，y 为 y 轴数据，数据可以列表或数组。
- **kwargs: 可选，用在二维平面图上，设置指定属性，如标签，线的宽度等。
- fmt: 可选，定义基本格式（如颜色、线条样式和标记），具体说明见下表。

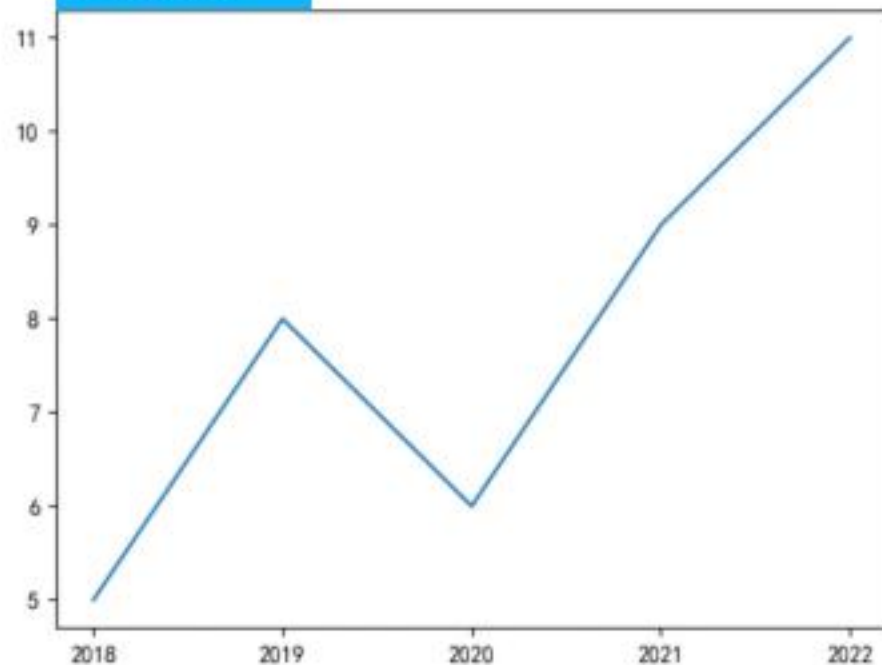
颜色	'b' 蓝色，'m' 洋红色，'g' 绿色，'y' 黄色，'r' 红色，'k' 黑色，'w' 白色，'c' 青绿色，'#008000' RGB 颜色字符串。多条曲线不指定颜色时，会自动选择不同颜色。
线条样式	'-' 实线，'--' 破折线，'-. ' 点划线，':' 虚线
标记字符	'.' 点标记，',' 像素标记(极小点)，'o' 实心圈标记，'v' 倒三角标记，'^' 上三角标记，'>' 右三角标记，'<' 左三角标记

示例2.3-1：通过Pyplot包中的函数实现2018到2022年5年的企业财务费用的可视化展示

示例2.3-1 代码

```
# 2-3 数据可视化任务 示例2.3-1  
import matplotlib.pyplot as plt #引入matplotlib.pyplot 库  
x = ["2018","2019","2020","2021","2022"] #年份列表  
y =[5,8,6,9,11] #各年份的财务费用列表，单位是百万  
plt.plot(x,y)  
plt.show()#显示图形
```

运行结果



plot() 函数全部参数使用说明，在jupyter中在函数前面输入? 后运行可得到函数的详细说明，该方法适用于全部函数获取函数使用说明。

示例代码

```
?plt.plot
```

运行结果

```
In [72]: plt.plot
```

```
In [ ]: 
```

```
Signature: plt.plot(*args, scalex=True, scaley=True, data=None, **kwargs)
```

```
Docstring:
```

```
Plot y versus x as lines and/or markers.
```

```
Call signatures::
```

```
plot([x], y, [fmt], *, data=None, **kwargs)
plot([x], y, [fmt], [x2], y2, [fmt2], ..., **kwargs)
```

```
The coordinates of the points or line nodes are given by *x*, *y*.
```

```
The optional parameter *fmt* is a convenient way for defining basic
formatting like color, marker and linestyle. It's a shortcut string
notation described in the *Notes* section below.
```

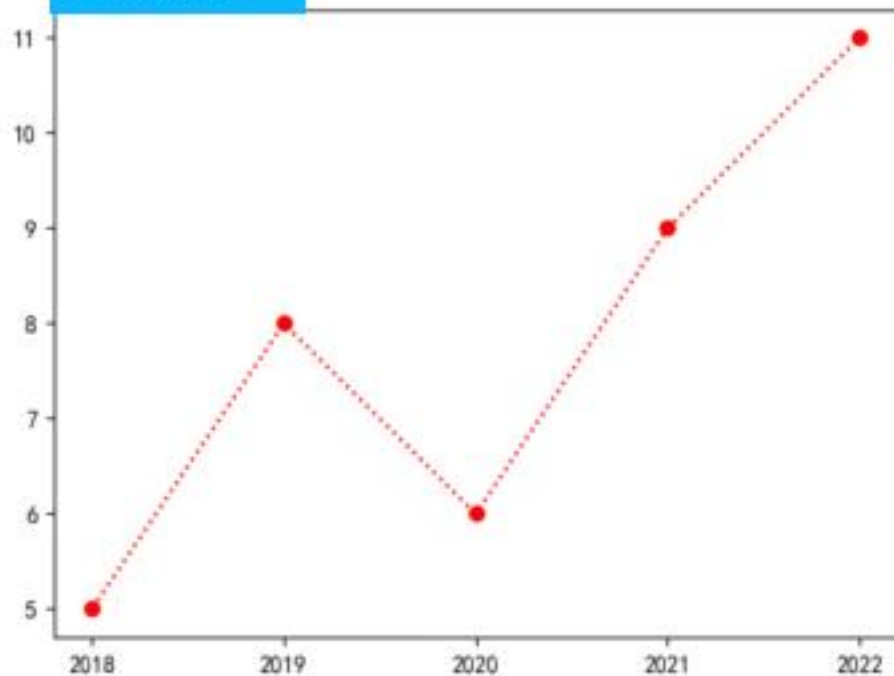
```
>>> plot(x, y)          # plot x and y using default line style and color
>>> plot(x, y, 'bo')     # plot x and y using blue circle markers
>>> plot(y)              # plot y using x as index array 0..N-1
>>> plot(y, 'r+')        # ditto, but with red plusses
```

示例2.3-2：通过Pyplot包中的函数实现2018到2022年5年的企业财务费用的可视化展示

示例2.3-2代码

```
# 2-3 数据可视化任务 示例2.3-2
import matplotlib.pyplot as plt #引入matplotlib.pyplot 库
x = ["2018","2019","2020","2021","2022"] #年份列表
y =[5,8,6,9,11] #各年份的财务费用列表，单位是百万
plt.plot(x,y,'r:o')
plt.show()#显示图形
```

运行结果

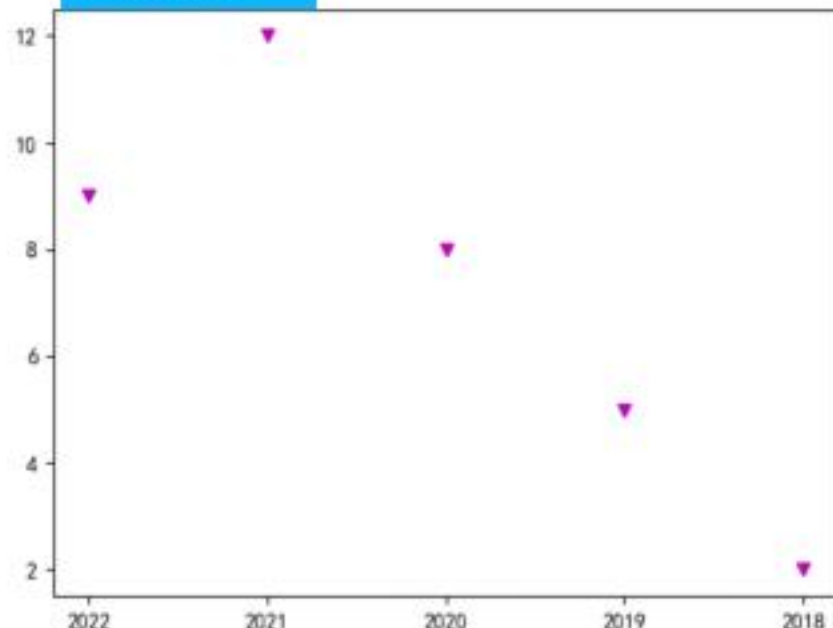


课堂练习1：根据A公司的流动负债数据和可视化图形，编写该图形实现的python代码。

数据

年份	流动负债（百万元）
2018	2
2019	5
2020	8
2021	12
2022	9

运行结果

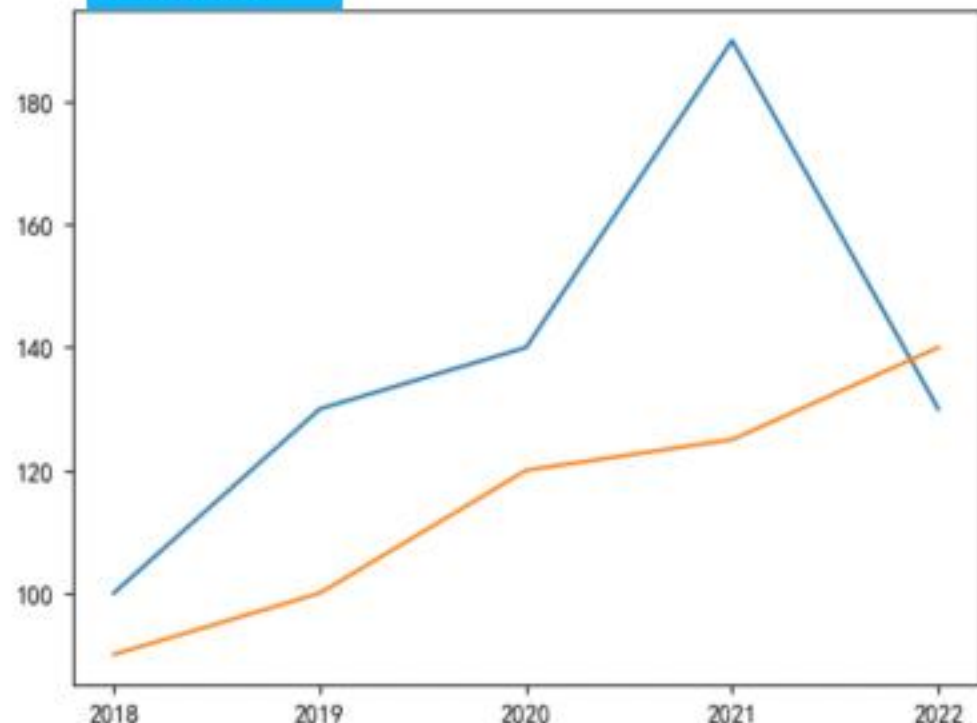


课堂练习2：根据AB两个公司的固定资产数据和对比的可视化图形，编写该图形实现的python代码。

数据

年份	A公司固定资产 (万元)	B公司固定资产 (万元)
2018	100	90
2019	130	100
2020	140	120
2021	190	125
2022	130	140

运行结果



在绘制图形的时候，为了确保图形的可读性，可以通过参数添加轴标签和标题，xlabel() 和 ylabel() 方法来设置 x 轴和 y 轴的标签，通过title() 方法来设置标题。

参数说明：

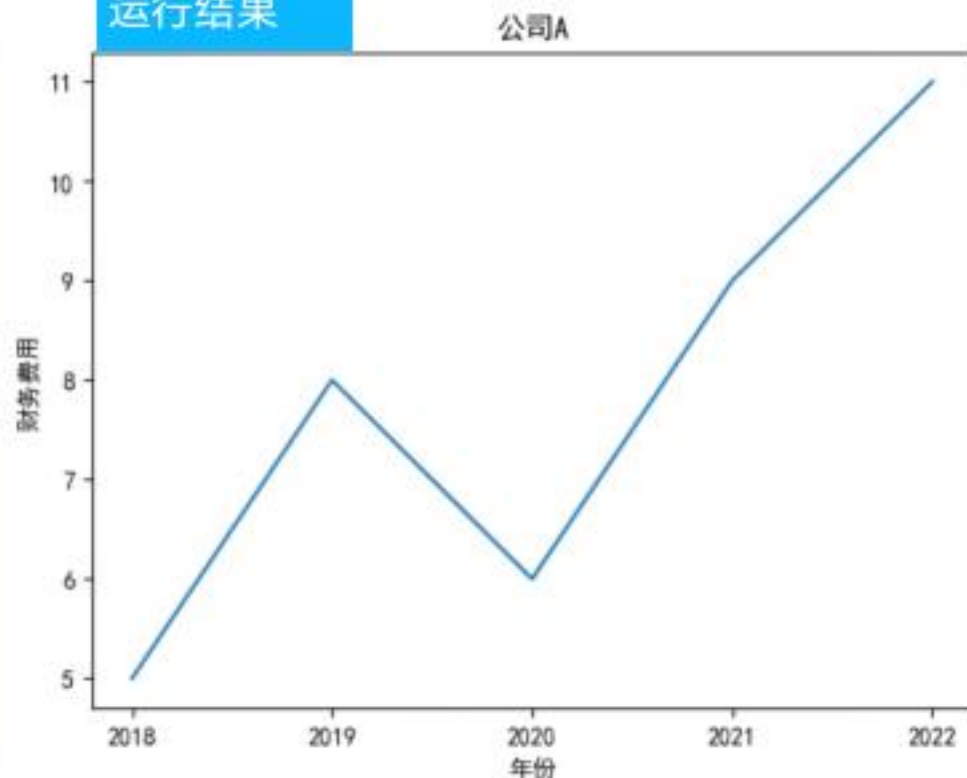
- title() 方法提供了 loc 参数来设置标题显示的位置，可以设置为: 'left', 'right', 和 'center', 默认值为 'center'。
- xlabel() 方法提供了 loc 参数来设置 x 轴显示的位置，可以设置为: 'left', 'right', 和 'center', 默认值为 'center'。
- ylabel() 方法提供了 loc 参数来设置 y 轴显示的位置，可以设置为: 'bottom', 'top', 和 'center', 默认值为 'center'。

示例2.3-3：通过Pyplot包中的函数实现2018到2022年5年的企业财务费用的可视化展示，要求设置横坐标、纵坐标和标题。

示例2.3-3代码

```
# 2-3 数据可视化任务 示例2.3-3
import matplotlib.pyplot as plt #引入matplotlib.pyplot 库
x = ["2018","2019","2020","2021","2022"] #年份列表
y =[5,8,6,9,11] #各年份的财务费用列表，单位是百万
plt.plot(x,y)
plt.xlabel("年份")
plt.ylabel("财务费用")
plt.title("公司A")
plt.show()#显示图形
```

运行结果



在设置图形的x、y轴标签和标题的时候，如果设置的是中文，可能会出现乱码的情况，这是因为Matplotlib 默认情况不支持中文，可以通过临时重写配置文件的方法，解决中文乱码的问题，代码如下：

解决中文乱码的代码

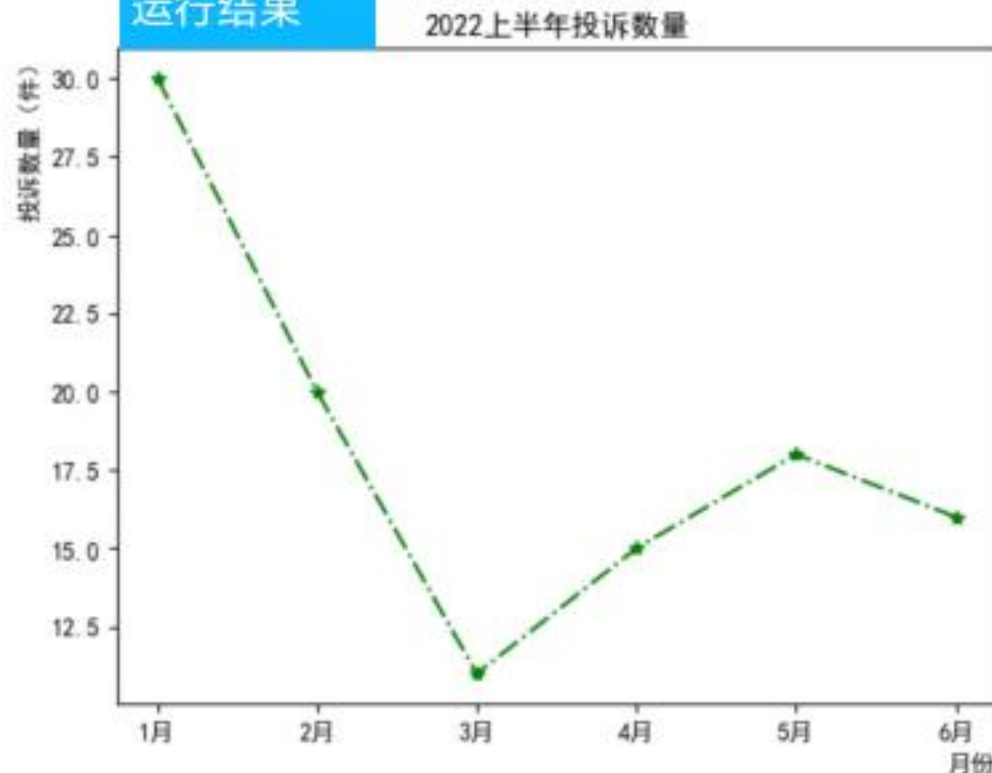
```
import matplotlib.pyplot as plt  
plt.rcParams["font.sans-serif"]=["SimHei"] #设置字体  
plt.rcParams["axes.unicode_minus"]=False #该语句解决图像中的“-”负号的乱码问题
```

课堂练习3：根据公司财务部统计业务部门对财务部的投诉数据和可视化图形，编写该图形实现的python代码。

数据

月份	投诉数量 (件)
1月	30
2月	20
3月	11
4月	15
5月	18
6月	16

运行结果



legend()函数可以在坐标轴上放置一个图例。该方法提供了loc参数设置图例的位置；提供了fontsize参数设置图例的字体大小，提供title参数设置图例的标题。

参数名称	设置语法	参数使用说明
Loc参数	plt.legend(loc='lower left')	0: 'best' (自动寻找最好的位置) 1: 'upper right' (右上角) 2: 'upper left' (左上角) 3: 'lower left' (左下角) 4: 'lower right' (右下角) 5: 'right' (右边中间) 6: 'center left' (左边中间) 7: 'center right' (右边中间) 8: 'lower center' (中间最下面) 9: 'upper center' (中间最上面) 10: 'center' (正中心)
Fontsize参数	plt.legend(fontsize=12)	设置字体大小，可以是int或者float
Title参数	plt.legend(title='图例标题')	设置图例的标题，内容给自定义

Text() 函数在坐标轴上根据坐标轴定位向数据添加文本，可用于设置坐标点的值，用于向数据坐标中位于x、y位置的坐标轴添加文本

text() 函数使用语法

```
pyplot.text(x, y, s, fontdict=None, withdash=False, **kwargs)
```

text() 函数常用参数说明：

- x, y: 显示内容的坐标位置
- s: 显示的内容
- fontdict: 一个定义s格式的dict
- fontsize: 字体大小
- color: str or tuple, 设置字体颜色, 单个字符候选项{ 'b' , 'g' , 'r' , 'c' , 'm' , 'y' , 'k' , 'w' }, 也可以' black' , 'red' 等。
- backgroundcolor: 字体背景颜色
- horizontalalignment(ha): 设置垂直对齐方式, 可选参数: left, right, center
- verticalalignment(va): 设置水平对齐方式, 可选参数: 'center' , 'top' , 'bottom' , 'baseline'
- rotation(旋转角度): 可选参数为: vertical, horizontal 也可以为数字
- alpha: 透明度, 参数值0至1之间

zip() 方法可以将传入的n个元素数量相等的列表压缩成一个元组，元组的每个元素就是一个n元组。

应用场景：zip()方法配合text()方法使用，通过遍历zip()方法的给坐标轴的点添加文本信息

遍历Zip() 方法

```
x = ["2018","2019","2020","2021","2022"]  
y =[5,8,6,9,11]  
xy = zip(x,y)  
for i,j in xy:  
    print(i,' ',j)
```

运行结果

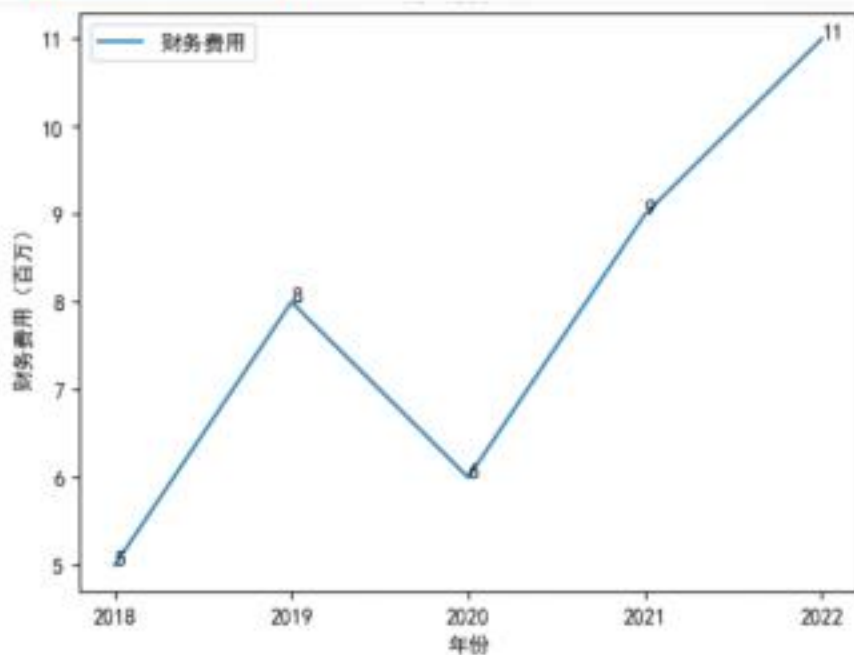
2018	5
2019	8
2020	6
2021	9
2022	11

示例2.3-4：通过Pyplot包中的函数实现2018到2022年5年的企业财务费用的可视化展示，要求设置图例并显示y轴的数据到图中。

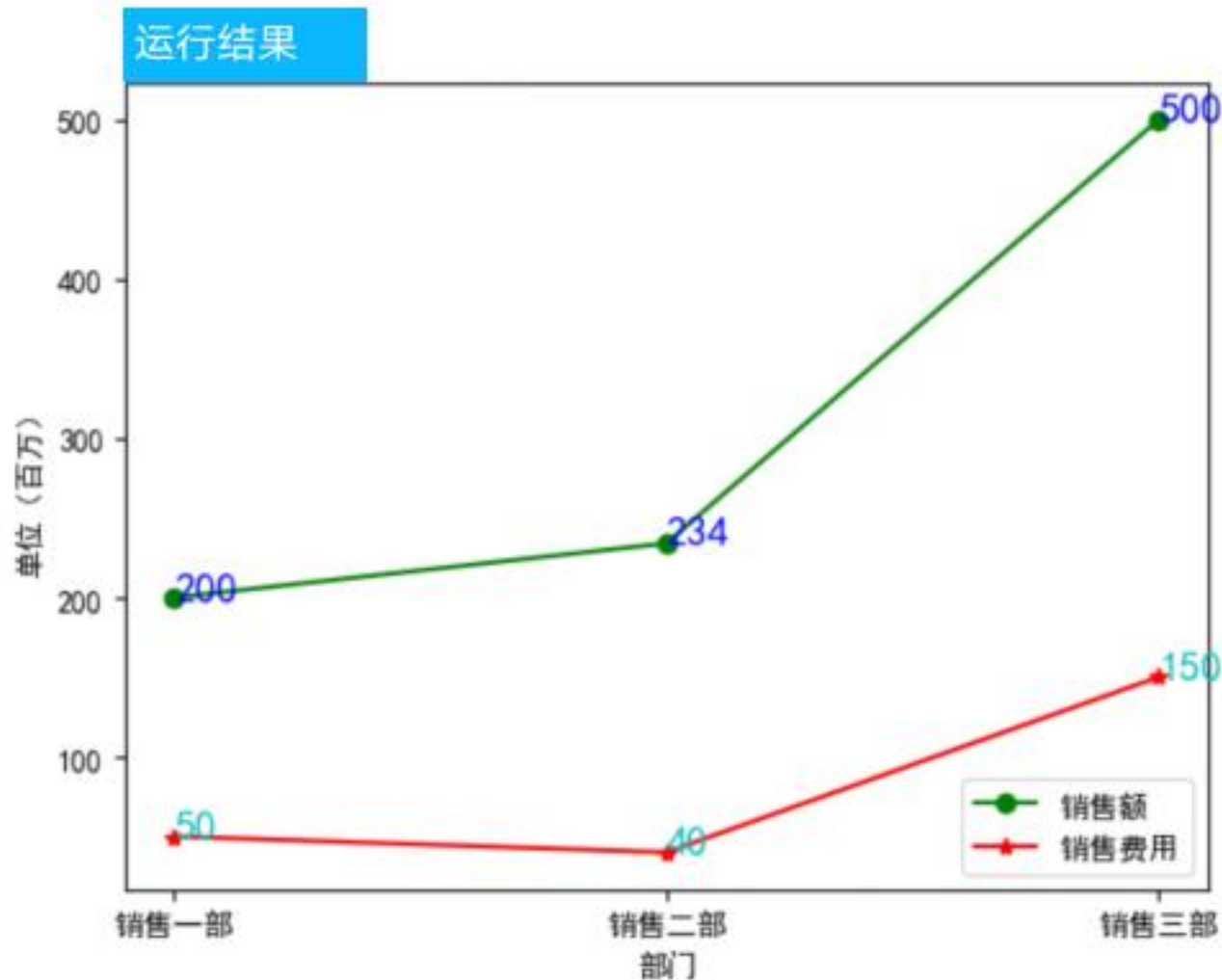
示例2.3-4代码

```
# 2-3 数据可视化任务 示例2.3-4
import matplotlib.pyplot as plt #引入matplotlib.pyplot 库
x = ["2018","2019","2020","2021","2022"] #年份列表
y =[5,8,6,9,11] #各年份的财务费用列表，单位是百万
for i,j in zip(x,y): #循环添加数据坐标中的文本
    plt.text(i,j)
plt.plot(x,y,label='财务费用') #设置线条的label参数
plt.xlabel("年份")
plt.ylabel("财务费用（百万）")
plt.title("公司的财务费用分析")
plt.legend() #显示图例
plt.show()#显示图形
```

运行结果



课堂练习4：根据公司销售额和销售费用可视化图形，编写该图形实现的python代码。



pie() 是绘制饼图的函数。

pie() 函数使用语法

```
matplotlib.pyplot.pie(x, explode=None, labels=None, colors=None, autopct=None, pctdistance=0.6,  
shadow=False, labeldistance=1.1, startangle=0, radius=1, counterclock=True, wedgeprops=None,  
textprops=None, center=0, 0, frame=False, rotatelabels=False, *, normalize=None, data=None)
```

pie() 函数常用参数说明：

- x: 浮点型数组，表示每个扇形的面积。
- explode: 数组，表示各个扇形之间的间隔，默认值为0。
- labels: 列表，各个扇形的标签，默认值为 None。
- colors: 数组，表示各个扇形的颜色，默认值为 None。
- autopct: 设置饼图内各个扇形百分比显示格式，%d%% 整数百分比，%0.1f 一位小数，%0.1f%% 一位小数百分比，%0.2f%% 两位小数百分比。
- labeldistance: 标签标记的绘制位置，相对于半径的比例，默认值为 1.1，如 <1 则绘制在饼图内侧。
- pctdistance: 类似于 labeldistance，指定 autopct 的位置刻度，默认值为 0.6。
- shadow: 布尔值 True 或 False，设置饼图的阴影，默认为 False，不设置阴影。
- radius: 设置饼图的半径，默认为 1。

pie() 函数常用参数说明(续):

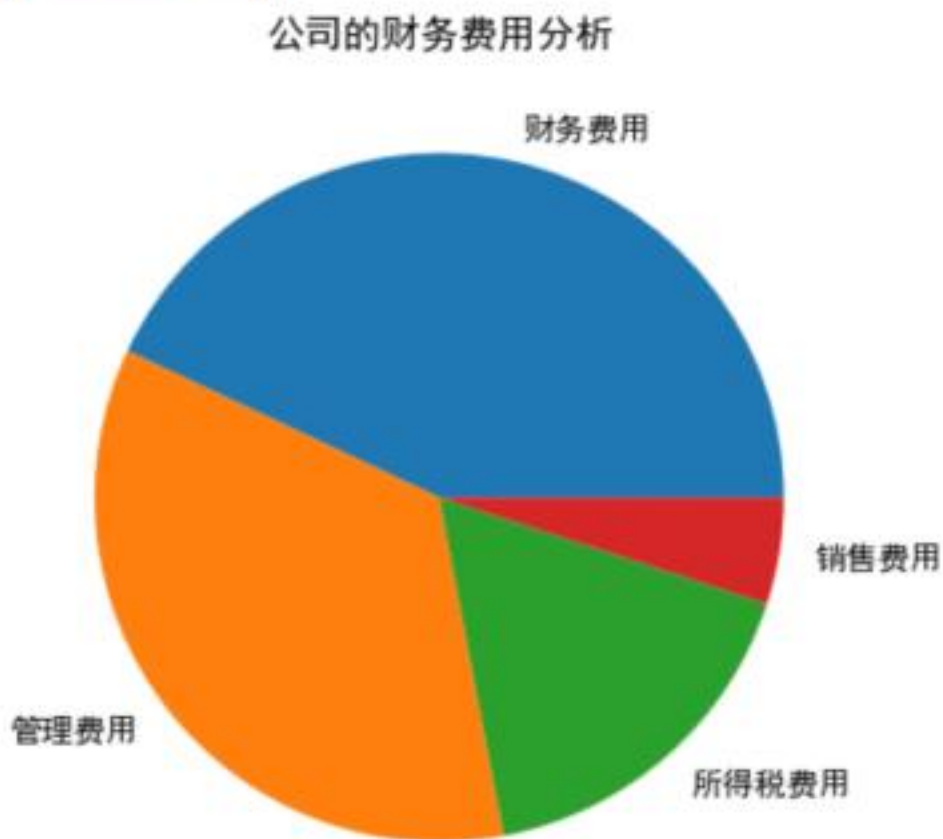
- startangle: : 起始绘制饼图的角度, 默认为从 x 轴正方向逆时针画起, 如设定 =90 则从 y 轴正方向画起。
- counterclock: 布尔值, 设置指针方向, 默认为 True, 即逆时针, False 为顺时针。
- wedgeprops : 字典类型, 默认值 None。参数字典传递给 wedge 对象用来画一个饼图。例如: wedgeprops={'linewidth':5} 设置 wedge 线宽为5。
- textprops : 字典类型, 默认值为: None。传递给 text 对象的字典参数, 用于设置标签 (labels) 和比例文字的格式。
- center : 浮点类型的列表, 默认值: (0,0)。用于设置图标中心位置。
- frame : 布尔类型, 默认值: False。如果是 True, 绘制带有表的轴框架。
- rotatelabels : 布尔类型, 默认为 False。如果为 True, 旋转每个 label 到指定的角度。

示例2.3-5：通过Pyplot包中的函数实现公司费用分析，要求用饼图展示。

示例2.3-5代码

```
# 2-3 数据可视化任务 示例2.3-5
import matplotlib.pyplot as plt #引入matplotlib.pyplot 库
y =[43,35,17,5] #费用金额（万元）
label=["财务费用","管理费用","所得税费用","销售费用"]
plt.pie(y,labels=label) #绘制饼图
plt.title("公司的财务费用分析")
plt.show()#显示图形
```

运行结果



课堂练习5：根据公司收入数据和可视化图形，编写该图形实现的python代码。

数据

项目	金额 (万元)
主营业务收入	678.92
其他业务收入	213.16
营业外收入	55.34

运行结果



bar() 是绘制柱状图的函数。

bar() 函数使用语法

```
matplotlib.pyplot.bar(x, height, width=0.8, bottom=None, *, align='center', data=None, **kwargs)
```

bar() 函数常用参数说明：

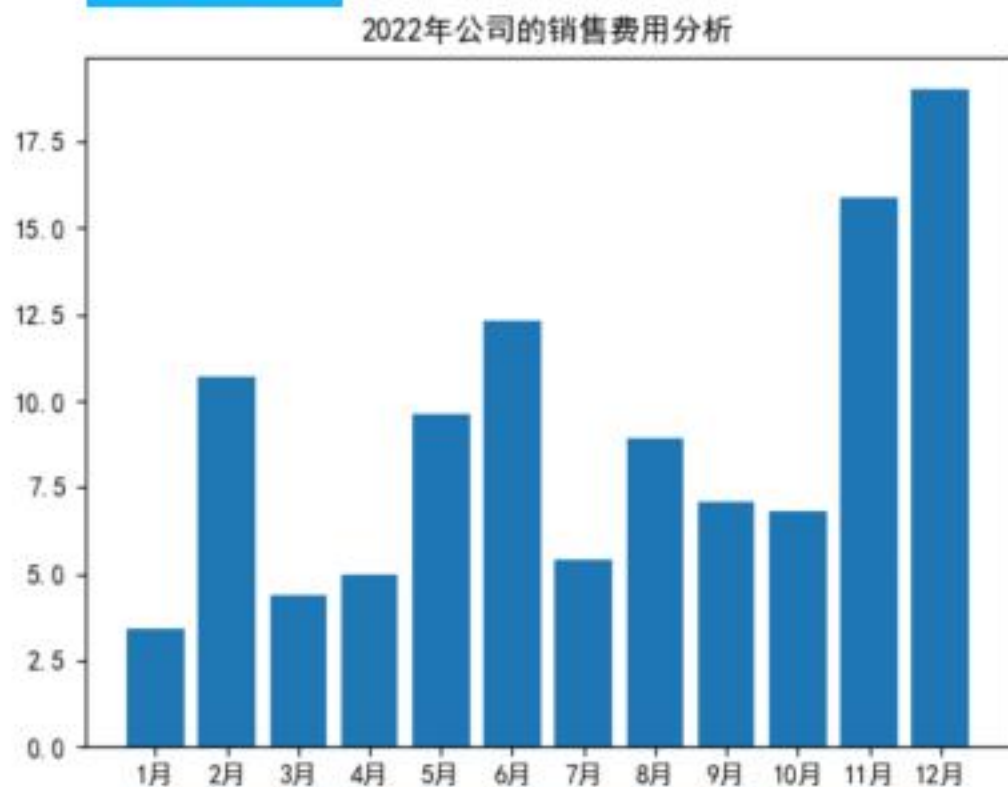
- x: 浮点型数组，柱形图的 x 轴数据。
- height: 浮点型数组，柱形图的高度。
- width: 浮点型数组，柱形图的宽度。
- bottom: 浮点型数组，底座的 y 坐标，默认 0。
- align: 柱形图与 x 坐标的对齐方式，'center' 以 x 位置为中心，这是默认值。'edge': 将柱形图的左边缘与 x 位置对齐。
- **kwargs: 其他参数。

示例2.3-6：通过Pyplot包中的函数实现公司2022年上半年的销售费用分析，要求用柱状图展示。

示例2.3-6代码

```
# 2-3 数据可视化任务 示例2.3-6
import matplotlib.pyplot as plt #引入matplotlib.pyplot 库
x = ["1月","2月","3月","4月","5月","6月","7月","8月","9月",
     "10月","11月","12月"] #月份
y =[3.4,10.7,4.4,5,9.6,12.3,5.4,8.9,7.1,6.8,15.9,19.01] #各
月份的销售费用列表，单位是万
plt.bar(x,y) #绘制柱状图
plt.title("2022年公司的销售费用分析")
plt.show()#显示图形
```

运行结果

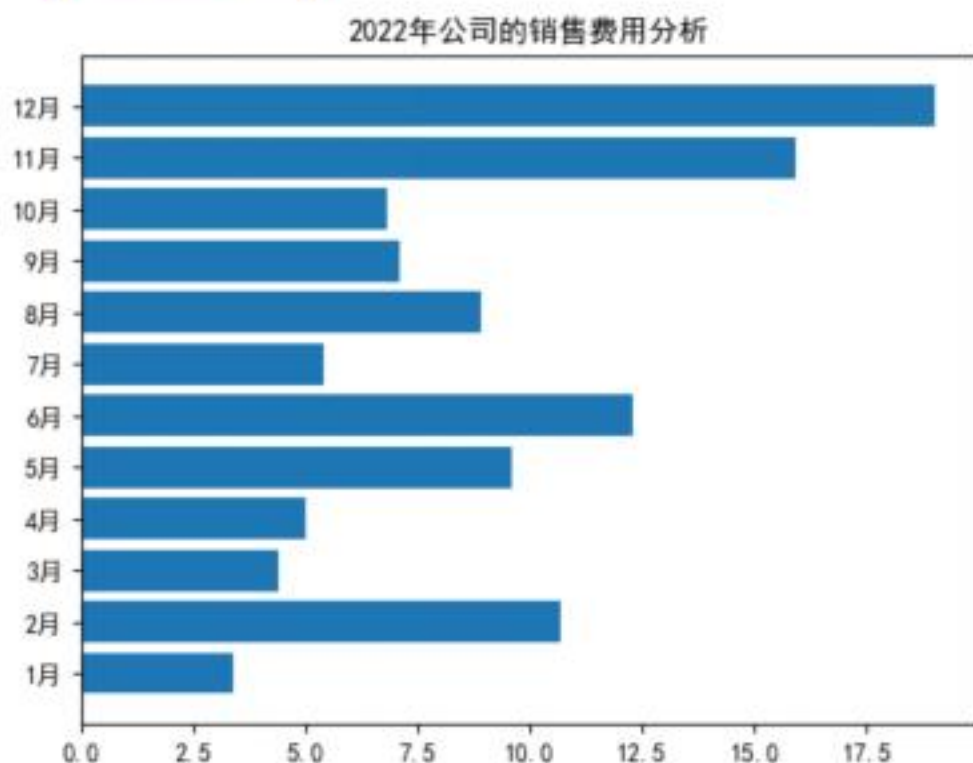


垂直方向的柱形图可以使用 barh() 方法来设置，以上示例2.3-6的代码中，将bar()换成barh() 后，运行后显示的结果如下：。

示例6代码

```
# 2-3 数据可视化任务 示例6
import matplotlib.pyplot as plt #引入matplotlib.pyplot 库
x = ["1月","2月","3月","4月","5月","6月","7月","8月","9月",
      "10月","11月","12月"] #月份
y =[3.4,10.7,4.4,5.9,6.1,12.3,5.4,8.9,7.1,6.8,15.9,19.01] #各
月份的销售费用列表，单位是万
plt.barh(x,y) #绘制柱状图
plt.title("2022年公司的销售费用分析")
plt.show()#显示图形
```

运行结果

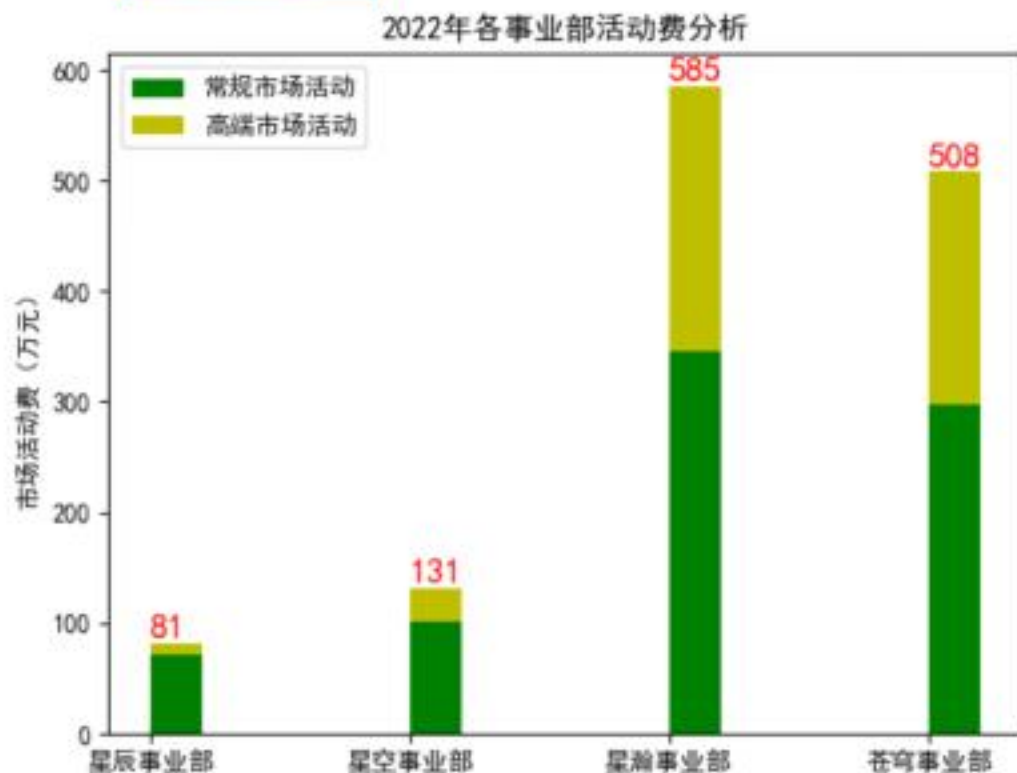


课堂练习6：根据公司各事业部的市场活动费用和分析图，编写该图形实现的python代码。

数据

部门	常规市场活动 (万元)	高端市场活动 (万元)
星辰事业部	71	10
星空事业部	101	30
星瀚事业部	345	240
苍穹事业部	298	210

运行结果



Pyplot的subplot () 方法来绘制多个子图，subplot() 方法在绘图时需要指定位置。

bar() 函数使用语法

```
subplot(nrows, ncols, index, **kwargs)
```

函数使用说明：

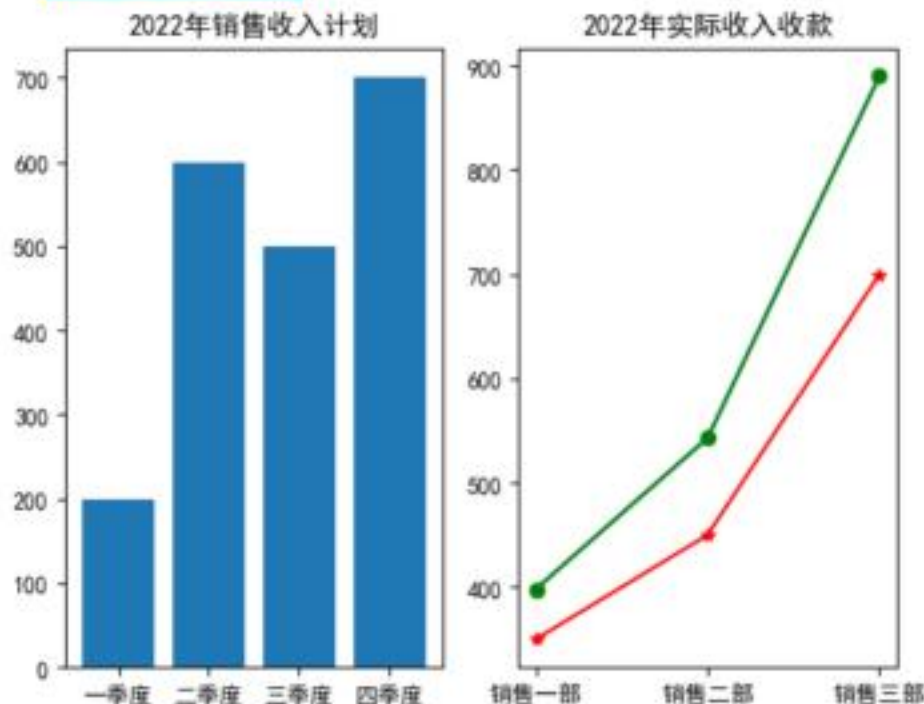
subplot函数将整个绘图区域分成 `nrows` 行和 `ncols` 列，然后从左到右，从上到下的顺序对每个子区域进行编号 1...N，左上的子区域的编号为 1、右下的区域编号为 N，编号可以通过参数 `index` 来设置，如设置 `numRows = 1`，`numCols = 2`，就是将图表绘制成 1x2 的图片区域，对应的坐标为(1, 1), (1, 2)，`index = 1`，表示的坐标为(1, 1)，即第一行第一列的子图，`plotNum = 2`，表示的坐标为(1, 2)，即第一行第二列的子图。

示例2.3-7：通过Pyplot包中的函数分析公司2022年的销售计划和销售实际数据，要求计划和实际数据分两个子图展示。

示例2.3-7代码

```
# 2-3 数据可视化任务 示例2.3-7
import matplotlib.pyplot as plt #引入matplotlib.pyplot 库
dept = ["销售一部","销售二部","销售三部"]
ay =[398,543,890] #销售收入（单位：万元）
by =[350,450,700] #销售收款（单位：万元）
quarter =["一季度","二季度","三季度","四季度"]
cy =[200,600,500,700] #计划销售收入（单位：万元）
plt.subplot(1, 2, 1)
plt.bar(quarter,cy)
plt.title("2022年销售收入计划")
plt.subplot(1, 2, 2)
plt.plot(dept,ay,'g-o',label='销售额')
plt.plot(dept,by,'r-*',label='销售费用')
plt.title("2022年实际收入收款")
plt.show()
```

运行结果



课堂练习7：根据公司采购部的采购金额和采购退款金额和分析图，编写该图形实现的python代码。

数据

部门	采购金额 (万元)	采购退货金额 (万元)
采购一部	340	40
采购二部	400	20
采购三部	370	43

运行结果



matplotlib.pyplot中还有很多本课程中没有介绍到的参数的使用和方法的使用，具体可以参考官网，官网中针对每一个方法都有详细的说明。

官网链接：https://matplotlib.org/stable/api/pyplot_summary.html



The screenshot shows the official matplotlib website's API summary for pyplot. The left sidebar lists various pyplot functions like `path`, `path_effects`, `pyplot`, `pyplot.acorr`, `pyplot.angle_spectrum`, `pyplot.annotate`, `pyplot.arrow`, `pyplot.autoscale`, `pyplot.axes`, `pyplot.axhline`, `pyplot.axhspan`, `pyplot.axis`, `pyplot.axline`, `pyplot.axvline`, `pyplot.axvspan`, `pyplot.bar`, `pyplot.bar_label`, `pyplot.barbs`, `pyplot.bart`, `pyplot.box`, and `pyplot.boxplot`. The main content area is titled **matplotlib.pyplot** and describes it as a state-based interface to matplotlib. It includes two code snippets: one for the implicit plotting style and another for the explicit object-oriented API using `Figure` and `Axis` objects. A right sidebar contains links to 'On this page', 'Plotting commands', 'Other commands', and 'Colormaps'. The footer mentions 'See Matplotlib Application Interfaces (APIs) for an explanation of the tradeoffs between the implicit'.

matplotlib.pyplot

matplotlib.pyplot is a state-based interface to matplotlib. It provides an implicit, MATLAB-like, way of plotting. It also opens figures on your screen, and acts as the figure GUI manager.

pyplot is mainly intended for interactive plots and simple cases of programmatic plot generation:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 5, 0.1)
y = np.sin(x)
plt.plot(x, y)
```

The explicit object-oriented API is recommended for complex plots, though pyplot is still usually used to create the figure and often the axes in the figure. See `pyplot.figure`, `pyplot.subplots`, and `pyplot.subplot_mosaic` to create figures, and Axes API for the plotting methods on an Axes:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 5, 0.1)
y = np.sin(x)
fig, ax = plt.subplots()
ax.plot(x, y)
```

See Matplotlib Application Interfaces (APIs) for an explanation of the tradeoffs between the implicit

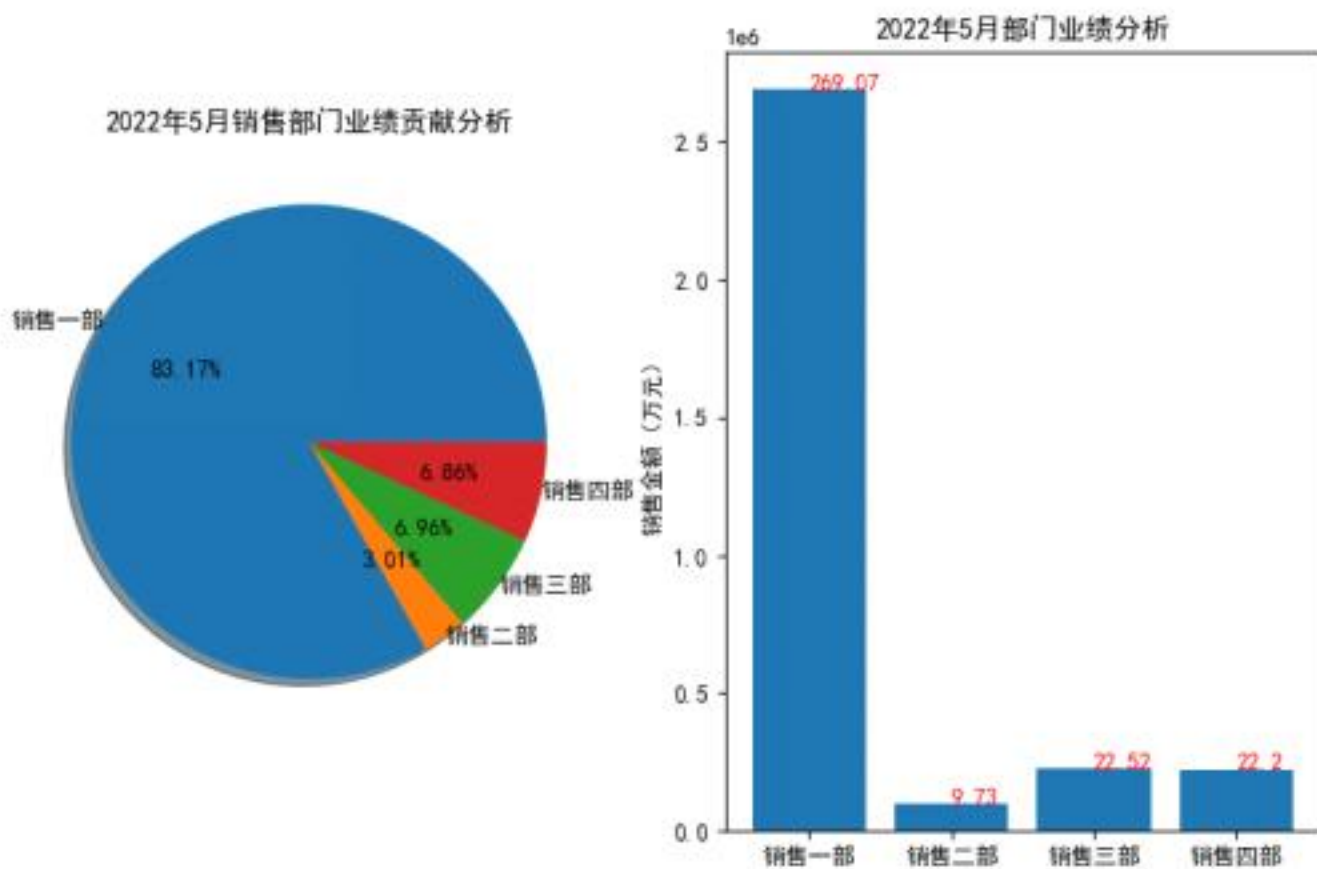
1 案例背景

2 知识准备

3 任务要求

4 任务解析

根据公司5月份的销售订单数据表《2-3-销售订单.xls》进行4个销售部门的业绩进行可视化分析，可以参考下图，也可自行实现。



2-3-销售订单

1 案例背景

2 知识准备

3 任务要求

4 任务解析

任务编码主思路：

- 1、通过pandas读取销售订单Excel文件的数据。
- 2、通过for循环按行处理销售订单数据，汇总各销售部门的销售业绩
- 3、调用pyplot的pie()和bar()函数绘制图像

任务实现代码：

参考教材中的代码答案



根据公司5月份的销售订单数据表《2-3-销售订单.xls》进行多个维度的业绩可视化分析，可以从商品维度、销售渠道、客户性质等维度进行分析。



感謝
ขอบคุณ
terima kasih
ありがとう
Thanks
谢谢